

MoodGarden

AI Mood & Habit Journal — Full Architectural & Engineering Blueprint

Document Purpose

This document serves as the comprehensive single-source implementation guide for the MoodGarden application, covering product requirements, full-stack architecture, relational database schema, API specifications, AI/sentiment analysis pipelines, design systems, and milestone task lists.

APPLICATION NAME

MoodGarden

BACKEND & AUTH

Supabase (PostgreSQL + RLS + GoTrue Auth)

PRODUCTION HOSTING

Vercel Edge • GitHub CI/CD

CORE ARCHITECTURE

Next.js 14+ (App Router) • TypeScript • Tailwind CSS

MACHINE LEARNING & AI

Hugging Face Inference API (DistilBERT SST-2)

DESIGN AESTHETIC

Calm Nature Palette • Fraunces & Inter Typography

1. Product Requirements Document (PRD)

1.1 Problem Statement

Most mood tracking applications record data points into sterile graphs without explaining the underlying causes. Users are presented with mood charts, but receive zero actionable insight into what habits drive their emotional states. MoodGarden bridges this gap by combining daily micro-journaling, habit correlation statistics, and a soothing, living calendar visualization.

1.2 Target User & Core Value

Designed for individuals seeking a 60-second daily reflection habit with an immediate visual reward: an organic, blooming garden tile reflecting their day, paired with earned correlation insights that uncover how habits (sleep, exercise, hydration, socializing) impact their mood.

1.3 Core User Journey

- 1. Authentication:** Seamless signup / sign-in with email and password.
- 2. Daily Journaling ("Today" View):** User logs a short reflection (max 500 characters), selects a 1–5 mood score, and checks completed habits.
- 3. Automated AI Analysis:** On submission, the entry is securely saved, and sentiment analysis runs asynchronously in the background.
- 4. The Garden Visual ("Garden" View):** Displays a monthly calendar grid where each day is represented by an organic colored tile (color matched to mood) and an SVG bloom icon scaled by completed habits. Clicking a tile opens a full day-detail modal.
- 5. Correlation Insights:** The app computes statistical habit-mood deltas over 14–30 entries and outputs plain-language insights (e.g., *"You've scored 1.3 points higher on mood on days you exercised."*).

1.4 Feature Scope & Non-Goals

Scope Category	Specifications
v1 Core (Must Have)	Email/password auth • Daily entry form (text, 1–5 mood, 5 standard habits) • One entry per day (same-day edit allowed) • AI sentiment scoring • Month garden view with bloom tiles • Day modal • Habit correlation card • Dark/Light mode • 375px mobile responsive.
v2 Scope (Post-v1)	Streak tracking counters • JSON/CSV export • Custom user-defined habits • Weekly email digest summary.
Explicit Non-Goals	No public profiles or social feeds • No native app store wrapper (PWA/responsive web only) • No offline sync complexity • No multi-language translation in v1.

2. System Architecture

2.1 Technology Stack

Layer	Technology	Architectural Rationale
Frontend Framework	Next.js 14+ (App Router, TypeScript)	Combines server-side rendering, client interactivity, and serverless API route handlers under a single unified deployment target.
Styling & Animation	Tailwind CSS + Framer Motion	Utility-first responsive styling with hardware-accelerated tile entrance animations and micro-interactions.
Database & Auth	Supabase (PostgreSQL + RLS + Auth)	Enterprise-grade relational database with JWT authentication and kernel-level Row Level Security policies.
AI Sentiment Model	Hugging Face Inference API	<code>distilbert-base-uncased-finetuned-sst-2-english</code> for fast, serverless sentiment classification without managing GPU infrastructure.
Hosting	Vercel	Seamless GitHub CI/CD integration, zero-config serverless route handlers, and edge caching.

2.2 Repository Directory Layout

```
moodgarden/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx      # Sign in interface
│   │   └── signup/page.tsx     # Registration interface
│   ├── today/page.tsx          # Daily entry submission form
│   ├── garden/page.tsx         # Visual month calendar grid
│   ├── api/
│   │   ├── entries/route.ts    # POST entry upsert, GET month entries
│   │   ├── entries/[date]/route.ts # GET single day detail
│   │   ├── sentiment/route.ts  # Internal Hugging Face sentiment handler
│   │   └── insight/route.ts    # Statistical correlation computation
│   ├── layout.tsx              # Root layout with fonts & theme provider
│   └── globals.css              # Tailwind base & CSS variables
├── components/
│   ├── EntryForm.tsx           # Combined journal form
│   ├── MoodPicker.tsx          # 5-point scale mood selector
│   ├── HabitChecklist.tsx      # Habit checkbox group
│   ├── GardenGrid.tsx          # 7-column calendar grid
│   ├── GardenTile.tsx          # Day tile with mood color & bloom icon
│   ├── DayModal.tsx            # Day inspection popup
│   ├── InsightCard.tsx         # Correlation sentence display
│   └── ThemeToggle.tsx         # Light/Dark mode switcher
├── lib/
│   ├── supabaseClient.ts       # Browser Supabase client
│   ├── supabaseServer.ts       # Server Supabase client (@supabase/ssr)
│   ├── sentiment.ts            # Hugging Face API interface
│   ├── correlation.ts           # Correlation statistics logic
│   └── moodColor.ts            # Mood score to color token mapper
├── types/
│   └── index.ts                 # TypeScript interfaces
├── middleware.ts                # Route protection via session check
├── .env.local.example           # Environment template
└── package.json
```

2.3 Data Flow & Async Handling

- 1. Entry Persistence Fast-Path:** Client dispatches `POST /api/entries`. The route handler immediately validates inputs and upserts the entry into Supabase, returning `200 OK` in `<150ms` so the user's interface never freezes.
- 2. Decoupled Sentiment Scoring:** Following persistence, sentiment calculation is triggered asynchronously without blocking the client. Even if Hugging Face experiences cold-starts, the journal entry is safely stored.
- 3. Garden Visualization:** `GET /api/entries?month=YYYY-MM` retrieves all entries for the active month, and client-side helpers map mood scores to gentle pastel tones.

3. Database Schema & Row Level Security

3.1 Table Definitions

The application relies on three core relational tables in PostgreSQL:

1. `entries` Table (One record per user per date)

Column	Type	Constraints & Defaults	Description
<code>id</code>	uuid	PRIMARY KEY, default <code>gen_random_uuid()</code>	Unique entry ID
<code>user_id</code>	uuid	NOT NULL, REFERENCES <code>auth.users(id)</code> ON DELETE CASCADE	Owning user
<code>entry_date</code>	date	NOT NULL	Calendar date of entry
<code>text</code>	text	NULLABLE (max 500 chars in app layer)	Journal content
<code>mood_score</code>	smallint	NOT NULL, CHECK (mood_score BETWEEN 1 AND 5)	Subjective rating (1–5)
<code>sentiment_score</code>	numeric(4,3)	NULLABLE	Normalized AI score (-1.0 to +1.0)
<code>created_at</code>	timestamptz	DEFAULT <code>now()</code>	Creation timestamp
<code>updated_at</code>	timestamptz	DEFAULT <code>now()</code>	Modification timestamp

Unique Constraint: `UNIQUE(user_id, entry_date)` ensures each user can have at most one journal entry per calendar day.

2. `habits` Table (Static reference catalog)

Column	Type	Notes
<code>key</code>	text, PRIMARY KEY	Unique identifier (e.g. <code>'exercise'</code>)
<code>label</code>	text, NOT NULL	Human-readable display title (e.g. <code>'Exercised'</code>)
<code>icon</code>	text	Display emoji or SVG name (e.g. <code>'🏃'</code>)

3. `habit_logs` Table (Many-to-many junction)

Column	Type	Notes
<code>id</code>	uuid, PRIMARY KEY	DEFAULT <code>gen_random_uuid()</code>
<code>entry_id</code>	uuid, NOT NULL	REFERENCES <code>entries(id)</code> ON DELETE CASCADE
<code>habit_key</code>	text, NOT NULL	REFERENCES <code>habits(key)</code>

3.2 Complete PostgreSQL Migration Script

```
create extension if not exists pgcrypto;

create table entries (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  entry_date date not null,
  text text,
  mood_score smallint not null check (mood_score between 1 and 5),
  sentiment_score numeric(4,3),
  created_at timestamptz default now(),
  updated_at timestamptz default now(),
  unique (user_id, entry_date)
);

create table habits (
  key text primary key,
  label text not null,
  icon text
);

create table habit_logs (
  id uuid primary key default gen_random_uuid(),
  entry_id uuid not null references entries(id) on delete cascade,
  habit_key text not null references habits(key)
);

-- Reference Seed Data
insert into habits (key, label, icon) values
  ('sleep', 'Slept well', '🌙'),
  ('exercise', 'Exercised', '🏃'),
  ('hydrate', 'Drank water', '💧'),
  ('socialize', 'Socialized', '👥'),
  ('eat_well', 'Ate well', '🍰')
on conflict (key) do nothing;

-- Performance Indexes
create index idx_entries_user_date on entries (user_id, entry_date desc);
create index idx_habit_logs_entry on habit_logs (entry_id);
```

3.3 Row Level Security (RLS) Policies

Security is enforced at the database engine level. Every client request must carry an authenticated JWT token corresponding to `auth.uid()`:

```

alter table entries enable row level security;

create policy "Users can view own entries"
  on entries for select
  using (auth.uid() = user_id);

create policy "Users can insert own entries"
  on entries for insert
  with check (auth.uid() = user_id);

create policy "Users can update own entries"
  on entries for update
  using (auth.uid() = user_id);

create policy "Users can delete own entries"
  on entries for delete
  using (auth.uid() = user_id);

alter table habit_logs enable row level security;

create policy "Users can view own habit logs"
  on habit_logs for select
  using (exists (
    select 1 from entries
    where entries.id = habit_logs.entry_id
    and entries.user_id = auth.uid()
  ));

create policy "Users can insert own habit logs"
  on habit_logs for insert
  with check (exists (
    select 1 from entries
    where entries.id = habit_logs.entry_id
    and entries.user_id = auth.uid()
  ));

create policy "Users can delete own habit logs"
  on habit_logs for delete
  using (exists (
    select 1 from entries
    where entries.id = habit_logs.entry_id
    and entries.user_id = auth.uid()
  ));

alter table habits enable row level security;
create policy "Anyone can read habits" on habits for select using (true);

```

4. API Specifications

4.1 POST /api/entries

Upserts today's entry (or specified date), synchronizes habit associations, and initiates async sentiment scoring.

Request Body:

```
{
  "entry_date": "2026-09-20",
  "text": "Productive day, went for an afternoon run and felt great.",
  "mood_score": 4,
  "habits": ["exercise", "hydrate"]
}
```

Response (200 OK):

```
{ "id": "4d9a16f2-b43e-4361-9c60-a23d8c1c5e40", "entry_date": "2026-09-20", "saved": true }
```

4.2 GET /api/entries?month=YYYY-MM

Fetches all entries logged by the authenticated user in the requested calendar month.

Response (200 OK):

```
{
  "entries": [
    {
      "entry_date": "2026-09-20",
      "mood_score": 4,
      "sentiment_score": 0.812,
      "text": "Productive day, went for an afternoon run and felt great.",
      "habits": ["exercise", "hydrate"]
    }
  ]
}
```

4.3 GET /api/entries/:date

Fetches full day details for a specific ISO date (used by `DayModal`). Returns `404 Not Found` if no entry exists.

4.4 POST /api/sentiment

Internal route to analyze journal text via Hugging Face and store normalized sentiment score in `entries.sentiment_score`.

```
// Request Body:
{ "entry_id": "4d9a16f2-...", "text": "Productive day, went for a run." }

// Response (200 OK):
{ "sentiment_score": 0.812 }

// Note: If timeout occurs (>5s), returns { "sentiment_score": null }
```

4.5 GET /api/insight

Retrieves user's last 30 entries, computes habit correlation deltas, and returns a natural language summary.

Response with sufficient data:

```
{
  "insight": "You've scored 1.3 points higher on mood on days you exercised, over your last 18 entries.",
  "habit": "exercise",
  "delta": 1.3,
  "sample_size": 18
}
```

Response when under sample threshold:

```
{ "insight": null, "reason": "not_enough_data" }
```

5. AI & Algorithmic Architecture

5.1 Sentiment Scoring Pipeline (`lib/sentiment.ts`)

Text reflections are analyzed using the DistilBERT model (`distilbert-base-uncased-finetuned-sst-2-english`). Hugging Face returns raw POSITIVE / NEGATIVE probabilities, which are mapped to a normalized float scale between **-1.000** (most negative) and **+1.000** (most positive).

```

export async function scoreSentiment(text: string): Promise<number | null> {
  if (!text || text.trim().length === 0) return null;

  try {
    const res = await fetch(
      "https://api-inference.huggingface.co/models/distilbert-base-uncased-finetuned-sst-2-english",
      {
        method: "POST",
        headers: {
          Authorization: `Bearer ${process.env.HUGGINGFACE_API_KEY}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify({ inputs: text }),
        signal: AbortSignal.timeout(5000), // 5-second timeout guard
      }
    );
    if (!res.ok) return null;
    const data = await res.json();
    const top = data[0]?.[0];
    if (!top) return null;
    return top.label === "POSITIVE" ? top.score : -top.score;
  } catch {
    return null; // Fail silently, never crash UI
  }
}

```

5.2 Correlation Engine (`lib/correlation.ts`)

A deterministic statistical engine calculates the mean mood difference for each habit across logged entries, enforcing a minimum sample size threshold (`MIN_SAMPLE = 7`) on both presence and absence sets to eliminate spurious noise.

```

export interface CorrelationResult {
  habit: string;
  delta: number;          // avg mood with habit - avg mood without
  sampleSizeWith: number;
  sampleSizeWithout: number;
}

const MIN_SAMPLE = 7;

export function computeStrongestCorrelation(
  entries: { mood_score: number; habits: string[] }[],
  habitKeys: string[]
): CorrelationResult | null {
  let best: CorrelationResult | null = null;

  for (const habit of habitKeys) {
    const withHabit = entries.filter(e => e.habits.includes(habit));
    const withoutHabit = entries.filter(e => !e.habits.includes(habit));

    if (withHabit.length < MIN_SAMPLE || withoutHabit.length < MIN_SAMPLE) continue;

    const avg = (arr: typeof entries) =>
      arr.reduce((sum, e) => sum + e.mood_score, 0) / arr.length;

    const delta = avg(withHabit) - avg(withoutHabit);

    if (!best || Math.abs(delta) > Math.abs(best.delta)) {
      best = {
        habit,
        delta: Math.round(delta * 10) / 10,
        sampleSizeWith: withHabit.length,
        sampleSizeWithout: withoutHabit.length,
      };
    }
  }
  return best;
}

```

6. Design System & Visual Specification

6.1 Color Palette & Design Tokens

MoodGarden is styled to feel peaceful and non-clinical, using warm off-whites, neutral organic dividers, and sage green accents.

Token	Light Mode Value	Dark Mode Value	Purpose
--bg	 #FAF7F2 (warm off-white)	 #1A1B1E	Page background
--surface	 #FFFFFF	 #232427	Card containers
--text-primary	 #2E2A26	 #EDEAE4	Headings & primary body
--text-muted	 #8A8378	 #9B978F	Secondary metadata
--accent	 #7C9885 (sage green)	 #8FB39B	Buttons & active rings
--border	 #EDE8E0	 #333437	Borders & dividers

6.2 Garden Tile Mood Scale

Calendar day tiles reflect mood without aggressive red/green grading tones:

Mood Score	Color Hex	Tone Description
1	 #D9C7B8	Muted clay (gentle low mood)
2	 #E3D4C2	Warm sand
3	 #D8DFC8	Neutral sage
4	 #BFDCC4	Soft mint
5	 #9FCBAE	Vibrant soft green (peak well-being)
Unlogged	 #EDE8E0	Empty outline / hollow border

6.3 Typography & Motion Principles

- **Headings:** `Fraunces` (warm serif via Google Fonts) provides an editorial, organic journal feel.
- **Body Text:** `Inter` delivers high legibility across mobile viewports.
- **Animations:** Staggered tile entry (20ms interval), smooth ease-out transitions (300–500ms), and strict adherence to `prefers-reduced-motion`.

7. Setup, Environment & Deployment Guide

7.1 Environment Variables Configuration

Store these keys in `.env.local` (strictly gitignored):

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-public-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-secret-key    # Server-side only
HUGGINGFACE_API_KEY=hf_yourHuggingFaceToken
```

7.2 Local Installation & Execution

```
# 1. Install dependencies
npm install

# 2. Start local development server
npm run dev

# 3. Access at http://localhost:3000
```

7.3 Vercel Deployment Workflow

1. Push codebase to GitHub main branch.
2. Link repository in Vercel dashboard.
3. Configure the 4 environment variables in Vercel Project Settings.
4. Trigger deployment; automated CI/CD pipeline builds and publishes preview & production edge releases.

8. Step-by-Step Implementation Tasks

Follow these 10 milestones in sequence to execute the complete build:

Milestone 1 — Project Scaffold

Foundation

- ☐ Initialize Next.js App Router project with TypeScript & Tailwind CSS
- ☐ Install `@supabase/supabase-js`, `@supabase/ssr`, `framer-motion`
- ☐ Configure `.env.local` and `.env.local.example`
- ☐ Apply design tokens to `tailwind.config.ts` and `globals.css`
- ☐ Import Google Fonts: Fraunces & Inter in `layout.tsx`

Milestone 2 — Database & Security

Backend

- ☐ Create Supabase project and database instance
- ☐ Execute full SQL migration: tables, habits seed, RLS policies, indexes
- ☐ Verify Row Level Security boundaries across simulated user sessions

Milestone 3 — Authentication

Auth

- ☐ Setup `lib/supabaseClient.ts` and `lib/supabaseServer.ts`
- ☐ Build `/login` and `/signup` authentication pages
- ☐ Implement `middleware.ts` route guard for `/today` and `/garden`
- ☐ Implement user sign-out workflow

Milestone 4 — Entry Flow

Core Features

- ☐ Create `MoodPicker.tsx`, `HabitChecklist.tsx`, and `EntryForm.tsx`
- ☐ Implement `POST /api/entries` and `GET /api/entries/:date`
- ☐ Connect `/today` page to query existing entry and persist updates

Milestone 5 — Garden Visualization

Visual Centerpiece

- ☐ Create `lib/moodColor.ts` color mapping function
- ☐ Implement `GET /api/entries?month=` month-filtering endpoint
- ☐ Construct `GardenGrid.tsx` and `GardenTile.tsx` with Framer Motion bloom animations
- ☐ Implement `DayModal.tsx` for inspection with month pagination

Milestone 6 — AI Sentiment Scoring

AI Pipeline

- ☐ Implement `lib/sentiment.ts` with DistilBERT API call & timeout safety
- ☐ Create `POST /api/sentiment` background scoring route
- ☐ Ensure async execution never blocks journal entry saves

Milestone 7 — Correlation Insights

Statistics

- ☐ Implement `lib/correlation.ts` statistical algorithm
- ☐ Create `GET /api/insight` endpoint with natural language formatter
- ☐ Build `InsightCard.tsx` component on `/garden` view

Milestone 8 — Polish & Accessibility

Polish

- ☐ Implement `ThemeToggle.tsx` for dark/light transitions
- ☐ Conduct 375px mobile responsive audit
- ☐ Configure skeleton loading states, empty states, and toast notifications

Milestone 9 — Deployment & Verification

Production

- ☐ Deploy application to Vercel and verify live production build
- ☐ Conduct end-to-end user verification across mobile and desktop devices
- ☐ Prepare complete project repository documentation and live demo

Milestone 10 — Real Usage

Impact

- ☐ Onboard 15–20 real users for 2-week journaling verification
- ☐ Record usage data, testimonials, and statistical correlation findings